

Horizon Validator

How Validation Works in the AI Refactoring Pipeline

What does the Validator do?

Whenever Horizon refactors your Java code, it does not just trust the AI got it right. The validator runs a series of checks to make sure the refactored code is correct, safe, and actually did what you asked. Think of it as a safety net that catches mistakes before they reach you.

The Big Picture: Four Security Gates

Every refactored piece of code must pass through four gates, in order:

Gate 1 — “Does it even compile?”

First, we check if the new code is valid Java. If it cannot be parsed, there is no point continuing. We try a few tricks (like wrapping bare statements in a class) because the AI sometimes outputs code fragments rather than full classes.

If it fails: We send it back to the AI with the syntax error and ask it to try again. It gets three chances. After that, we tell the Planner to come up with a different strategy.

Gate 2 — “Did complexity get worse?”

Some refactorings should make code simpler. We use **cyclomatic complexity** (a standard metric that counts how many different paths through a piece of code there are — higher numbers mean harder-to-test code).

If it fails: The system says “your refactoring made the code harder to understand” and asks the AI to try a different approach.

Gate 3 — “Did you touch code you should not have?”

When you ask Horizon to refactor a specific method, the AI should only change that method. Everything else should stay exactly as it was. The validator compares the structure of every non-target method between the original and refactored code.

This gate applies the same way to every refactoring type. There are no per-refactoring rules — the check is universal. If a method outside the target scope changed structurally, that is a boundary violation. The system rejects the refactoring.

If it fails: The AI is told exactly which method was accidentally modified and asked to preserve it.

Gate 4 — “Did the refactoring actually happen?”

This is the most specific check. For each type of refactoring, the validator checks that the expected structural change occurred. For example:

If it fails: The system tells the AI “you said you extracted a method but the method count did not change” and asks it to try again.

The 12 Refactoring Types and What They Mean

Refactoring	Category	What It Does
Flatten Conditional	Control Flow	Transforms deeply nested if-statements into a flat sequence of early returns or guard clauses. Turns “arrow code” into linear, top-to-bottom flow.
Decompose Conditional	Control Flow	Breaks a complicated boolean expression into separate named boolean variables. Each variable captures one business condition, making the logic self-documenting.
Consolidate Conditional	Control Flow	Merges multiple if-statements or switch cases that share identical behavior into a single condition or polymorphic dispatch.
Remove Control Flag	Control Flow	Eliminates the “boolean sentinel” pattern — a flag variable used to control a loop — in favor of direct break or return statements.
Replace Loop with Pipeline	Control Flow	Converts an imperative for-loop into a declarative Java Stream pipeline using <code>.stream()</code> , <code>.filter()</code> , <code>.map()</code> , <code>.collect()</code> .
Split Loop	Control Flow	Takes one loop that performs two unrelated tasks and splits it into two separate, single-purpose loops.
Extract Method	Method Movement	Moves a block of code into a new, descriptively named helper method. The original location calls the new method instead.
Inline Method	Method Movement	The reverse of extraction — takes a method whose body is simple and self-explanatory, replaces every call site with the body directly, then removes the method.
Extract Variable	State Management	Replaces a repeated or complex expression with a named variable computed once. The variable name explains what the expression represents.
Inline Variable	State Management	Replaces a variable that just holds a simple expression result with the expression itself, when the variable name adds no clarity.
Extract Constant	State Management	Promotes a hardcoded magic number or literal string into a named static final field. Documents what the value means and centralizes it.
Rename Symbol	State Management	Changes the name of a variable, method, or field to be more descriptive — without altering any behavior. Pure renaming with zero logic change.

Cyclomatic Complexity Rules Per Refactoring Type

CC measures how many independent paths exist through code. Some refactorings naturally add or remove paths. The validator uses four different rules depending on the refactoring type:

Refactoring	CC Rule	What This Means
Flatten Conditional	Loosened	Complexity can increase by up to 1. Early returns add an extra exit path, so a small CC bump is expected and acceptable.
Decompose Conditional	Extract Rule	The target method's CC must not increase. Adding named variables should not change complexity, and the refactoring should not introduce new branches.
Consolidate Conditional	Strict	Overall CC must not increase. Merging branches should reduce or maintain complexity — never add more paths.
Remove Control Flag	Strict	Overall CC must not increase. Removing a flag should eliminate the branching around it, not add new ones.
Replace Loop w/ Pipeline	Strict	Overall CC must not increase. Streams inherently flatten control flow, so CC should go down or stay flat.
Split Loop	Loosened	Complexity can increase by up to 1. Having two loops instead of one adds a new path, a reasonable tradeoff for clarity.
Extract Method	Extract Rule	The source method's CC must not increase. Extracting code should simplify the caller. The new helper method's CC is not restricted.
Inline Method	Skip	No complexity check. Inlining naturally makes the caller more complex as it absorbs the inlined logic, so CC increase is expected.
Extract Variable	Strict	Overall CC must not increase. Adding a variable declaration does not change the number of execution paths.
Inline Variable	Strict	Overall CC must not increase. Removing a variable does not add or remove execution paths.
Extract Constant	Strict	Overall CC must not increase. Adding a constant field has no impact on control flow.
Rename Symbol	Strict	Overall CC must not increase. Renaming should change nothing but the name — all structure must be preserved.

How the CC Rules are Enforced

- **Strict:** If the refactored code's overall CC is higher than the original, the refactoring is rejected.
- **Loosened:** Same as strict, but allows the refactored CC to exceed the original by up to 1.
- **Extract Rule:** Instead of checking overall code complexity, it checks only the specific target method. If that method's CC went up, the refactoring is rejected. Also rejected if the target method cannot be found in the refactored code at all.
- **Skip:** The complexity gate is bypassed entirely for this refactoring type.

Intent Check Requirements Per Refactoring Type

Each refactoring type has a custom structural check that verifies the expected change actually occurred in the code.

Refactoring	What is Checked	Pass Condition	Fail Condition
Flatten Cond.	Max nesting depth of IfStatement nodes	New depth is strictly less than old depth	Depth unchanged or increased
Decompose Cond.	BinaryOperation count + new variables in conditionals	Binary ops decreased, OR new variables introduced and used in a conditional	No new variables AND no binary-op reduction
Consolidate Cond.	Total IfStatement + SwitchStatement count	Count decreased	Count unchanged or increased
Remove Ctrl Flag	Break/Return count + variable changes	Any one of three: (1) exits increased, (2) variables removed, (3) new variables with existing exits	No change in exits, no variables removed or added
Replace Loop w/ Pipe.	Loop count + Stream API evidence	Loop count decreased AND stream evidence found (or loop decreased as fallback)	Loop count unchanged
Split Loop	ForStatement + WhileStatement count	Count increased	Count unchanged or decreased
Extract Method	MethodDeclaration count	Count increased	Count unchanged or decreased
Inline Method	MethodDeclaration count	Count decreased or stayed the same	Count increased
Extract Variable	VariableDeclarator count	Count increased	Count unchanged or decreased
Inline Variable	VariableDeclarator count	Count decreased or stayed the same	Count increased
Extract Constant	FieldDeclaration count + UPPERCASE names	Field count increased, OR new uppercase-named variables appeared	No new fields and no new uppercase variables
Rename Symbol	Per-method structural signatures (SHA-256 of AST skeleton)	Every original method's signature has a match in the refactored code	At least one original method's signature has no match

Boundary Preservation

This gate has no per-refactoring rules. It works the same way for every refactoring type.

The validator extracts all methods from both the original and refactored code, then compares them side by side. Any method that exists in both versions but is not in the target scope must have an identical structural signature. If a non-target method's structure changed, the refactoring is rejected.

New methods are always allowed — the check only applies to methods that already existed. Adding new classes, enums, or helper methods as part of the refactoring strategy is fine. Modifying something that was supposed to stay untouched is not.

How the System Handles Failure

When a gate fails, Horizon does not just give up. It follows a recovery strategy:

1. **Syntax failure:** Send the error back to the AI Generator and ask it to fix the syntax. Up to 3 attempts.
2. **Structural failure** (complexity, boundary, or intent): Send the finding back to the Generator once for a targeted fix.
3. **Still failing after that:** Escalate to the Planner to rethink the entire refactoring strategy. Up to 3 strategy retries.

If all retries are exhausted, the system reports `ABORT_SYNTAX` or `ABORT_STRATEGY` and returns the best effort so far.

Error Tiers (Severity Levels)

Failures are organized into five tiers:

Tier	Name	What Went Wrong
Tier 1	Syntax	Generated code is not valid Java — cannot even parse it
Tier 2-A	Complexity	Cyclomatic complexity increased beyond what is allowed
Tier 2-B	Boundary	The AI modified a method it was not supposed to touch
Tier 2-C	Intent	The refactoring did not produce the expected structural change
Tier 3	Judge	The Judge model reviewed the result and rejected it

Tier 1 is the most fundamental — if code does not parse, nothing else matters. Tier 3 is the final human-judgment-like review performed by a separate AI model.

Real Example: What Happens During Validation

Let us say you ask Horizon to extract a method from this code:

```
class Calculator {
    void process() {
        int total = price * quantity;
        double tax = total * 0.08;
        System.out.println(total + tax);
    }
}
```

The AI might produce:

```
class Calculator {
    void process() {
        double result = calculateTotal(price, quantity);
        System.out.println(result);
    }
    double calculateTotal(int price, int quantity) {
        int total = price * quantity;
        double tax = total * 0.08;
        return total + tax;
    }
}
```

The validator then checks:

4. **Syntax:** Parses OK — both are valid Java.
5. **Complexity:** Uses the Extract Rule — the process() method's CC must not increase. Original had 1 path, refactored has 1 path. Pass.
6. **Boundary:** No other methods exist to leak into. Pass.
7. **Intent:** Method count was 1, now it is 2. Extract-method detection passes.

All gates green → the refactoring is accepted and moves on to the Judge for final review.

Where to Find the Code

- **Core validator logic:** `backend/app/modules/validator/__init__.py`
- **Phase 4 orchestration:**
`backend/app/modules/orchestrator/phases/phase4_validation.py`
- **Types and data structures:** `backend/app/utis/types.py` — RefactorIntent, FailureTier, StructureUnit
- `backend/app/utis/schemas.py` — ValidationFinding, ErrorReport, IntentPacket